# Final Project Report: Autonomous Firefighting Robot Simulation in ROS 2

**Submitted By:**

| Name | Roll No. |
|---|---|
| Arin Goyal | 102323006 |
| Avtesh Singh | 102323016 |
| Parth Chhabra | 102373007 |

**Submitted To:** Dr. Raja Rout

**Subject:** Robotics Systems Simulation (URA601)

**Framework:** ROS 2 (Humble) / Gazebo

---

## DECLARATION

We hereby declare that the project work titled "Mobile Robot Simulation and Vision-Based Object Tracking using ROS 2" is an authentic record of our own work carried out at Thapar Institute of Engineering and Technology as a requirement for the award of the degree of B.E. in Mechanical Engineering.

This project has been carried out under the guidance of Dr. Raja Rout (Professor, MED, TIET) during the academic session from January 2026 to May 2026.

The work presented in this report has not been submitted elsewhere for the award of any other degree or diploma.

**Date:** 05.05.2026
**Place:** Patiala

# ACKNOWLEDGEMENT

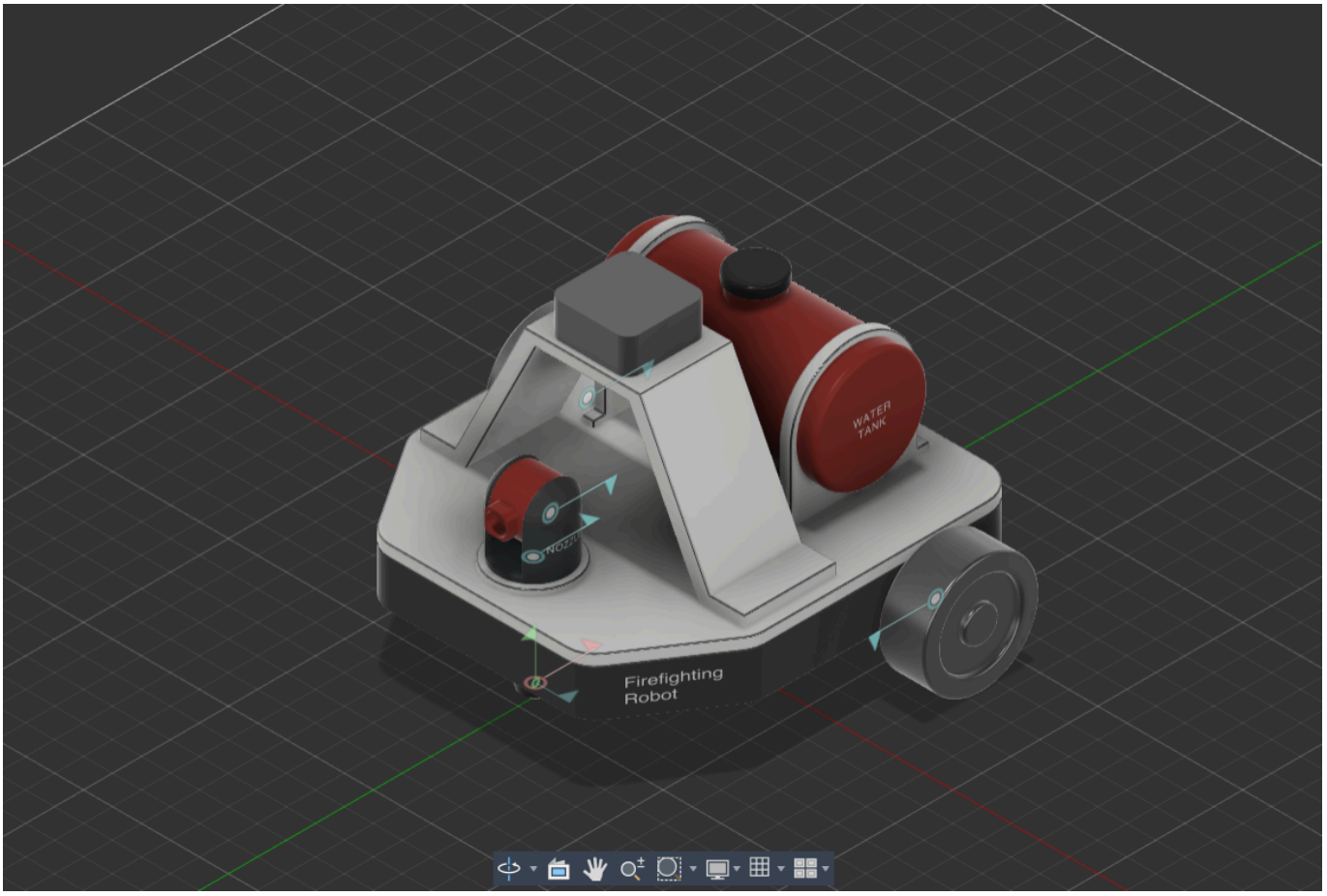
We would like to express our sincere gratitude to our project guide for their continuous support and guidance throughout this project. Their insights helped us understand ROS 2, simulation, and autonomous navigation systems effectively.

We also thank our teammates for their collaboration and consistent efforts.

# Table of Contents


1. Abstract
2. Introduction
    - 2.1 Project Objectives
    - 2.2 Tools and Technologies
3. Methodology
    - 3.1 Robot Description — URDF/Xacro
    - 3.2 Gazebo and ros2_control Configuration
    - 3.3 Autonomous PID Control Execution Node
    - 3.4 Launch Architecture
4. ROS 2 Computation Graph (rqt_graph)
    - 4.1 Active Nodes
    - 4.2 Key Topics and Actions
5. TF2 Transform Tree
    - 5.1 Frame Hierarchy
    - 5.2 Frame Details
    - 5.3 TF2 in the Mapping Pipeline
6. Results and Outputs
    - 6.1 System Startup Verification
    - 6.2 PID Execution Output
    - 6.3 SLAM Mapping Output
    - 6.4 Issues Encountered and Resolved
7. Project Structure
8. Conclusion
9. References

# 1. Abstract




This project presents the design, simulation, and control of a differential-drive firefighting mobile robot built using ROS 2 Humble. The primary objective is to implement an autonomous pipeline in which the robot drives to a precise target distance while simultaneously aiming its water nozzle—all driven by custom mathematical control algorithms without relying on pre-built navigation stacks.

The robot is described using URDF/Xacro and simulated in Gazebo. The control architecture comprises a `diff_drive` plugin for chassis movement and a layered `ros2_control` stack for nozzle articulation. Motion logic is handled by a custom dual-loop Proportional-Integral-Derivative (PID) Python node. Furthermore, a 2D LiDAR scanner is integrated to achieve Simultaneous Localization and Mapping (SLAM) using the `slam_toolbox`, allowing the robot to autonomously generate an occupancy grid of a simulated warehouse.

# 2. Introduction

Firefighting robots mitigate human risk by autonomously navigating hazardous environments to extinguish fires. Implementing this in a simulation before physical deployment allows safe iteration, tuning of control algorithms, and debugging without hardware risk.

ROS 2 (Robot Operating System 2) is the industry-standard middleware for modern robotics. It uses DDS for real-time distributed communication. This project builds a complete software stack for a custom firefighting robot from scratch: URDF robot description, Gazebo physics configuration, custom PID motion control, and 2D SLAM mapping.

## 2.1 Project Objectives

- Model a differential-drive firefighting robot with a motorized nozzle in URDF/Xacro.
- Configure Gazebo plugins for physics, 2D LiDAR, and differential drive odometry.
- Set up `ros2_control` for hardware abstraction of the nozzle joint.
- Implement a Python node to execute autonomous driving and aiming via dual-loop PID control.
- Verify mapping and localization by generating a 2D occupancy grid using `slam_toolbox`.

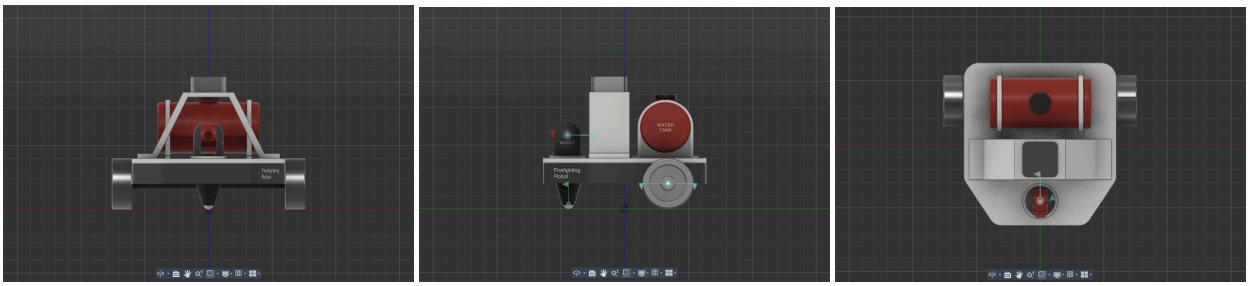
### 2.2 Tools and Technologies

| Tool / Library | Purpose |
|---|---|
| ROS 2 Humble | Core middleware — nodes, topics, services, actions |
| Gazebo | 3D physics simulation environment & sensor generation |
| ros2_control | Hardware abstraction — joint state & velocity control for the nozzle |
| slam_toolbox | 2D mapping and SLAM algorithm |
| URDF / Xacro | Robot description — geometry, joints, inertia, collisions |
| RViz2 | 3D visualisation — robot model, laser scans, TF tree, occupancy grid |
| Python (rclpy) | Custom node execution, PID mathematical logic, topic publishing |

## 3. Methodology

The implementation follows a bottom-up layered approach. Each layer was built and validated independently before integration with the layer above it.

### 3.1 Robot Description — URDF/Xacro



The robot is defined in `final_assembly.xacro` with multiple links and joints. The kinematic chain is:

```
world → map → odom → base_footprint → dummy → base_link
                                              ├── left_wheel
                                              ├── right_wheel
                                              ├── caster_wheels
                                              ├── nozzle_base → nozzle
                                              └── lidar_link
```

The wheel joints are `continuous`, while the nozzle joint is `revolute`. The `ros2_control` hardware interface exposes a velocity command interface for the nozzle, allowing direct angular velocity manipulation.

### 3.2 Gazebo and ros2_control Configuration

Two primary paradigms handle the physics and actuator control:

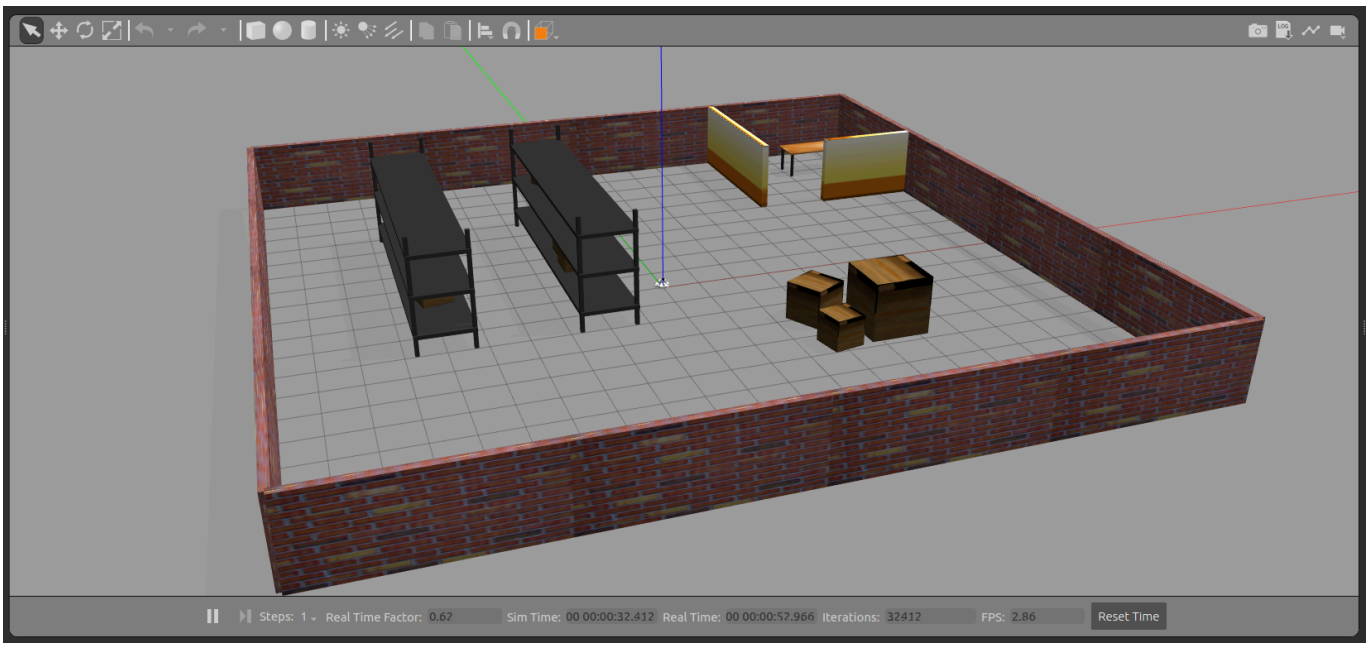
| Plugin / Controller | Type | Purpose |
|---|---|---|
| diff_drive | Gazebo Plugin | Translates `/cmd_vel` to wheel torques, publishes `/odom` |
| joint_state_broadcaster | JointStateBroadcaster | Publishes `/joint_states` at 50 Hz |
| nozzle_velocity_controller | JointGroupVelocityController | Commands nozzle pitch velocity via topic |

### 3.3 Autonomous PID Control Execution Node

The `pid_controller.py` node runs a dual-loop Proportional-Integral-Derivative (PID) algorithm to handle complex, simultaneous tasks.

| Loop | Target Goal | Sensor Input | Output Mechanism |
|---|---|---|---|
| **Drive PID** | Drive exactly 2.0 meters forward | Euclidean distance derived from `/odom` | Publishes `linear.x` on `/cmd_vel` |
| **Aim PID** | Tilt nozzle upwards by 0.5 radians | Real-time joint angle from `/joint_states` | Publishes Float array on `/nozzle_velocity_controller/commands` |

### 3.4 Launch Architecture




The `gazebo.launch.py` file orchestrates the full system. Nodes are spawned sequentially to avoid race conditions.

```
t = 0s  gazebo_server          → launches warehouse.world with physics
t = 0s  gazebo_client          → launches Gazebo GUI
t = 0s  robot_state_publisher  → publishes robot_description & static TF2 frames
t = 1s  spawn_entity           → injects the URDF model into Gazebo
t = 2s  spawner                → joint_state_broadcaster
t = 3s  spawner                → nozzle_velocity_controller
```

## 4. ROS 2 Computation Graph (rqt_graph)

The ROS 2 computation graph shows the active nodes and the topics connecting them during autonomous operation and mapping.

### 4.1 Active Nodes

| Node Name | Role in System |
|---|---|
| `/gazebo` | Simulates physics, acts as the hardware interface, publishes LiDAR and Odometry |
| `/robot_state_publisher` | Subscribes to `/joint_states`; publishes all TF2 link transforms |
| `/controller_manager` | Manages `ros2_control` lifecycle — configure, activate, deactivate |
| `/pid_controller` | High-level logic — calculates PID math, publishes `/cmd_vel` and nozzle commands |
| `/slam_toolbox` | Consumes `/scan` and `/odom` to generate the global `/map` |

### 4.2 Key Topics and Actions

| Topic | Message Type | Publisher → Subscriber |
|---|---|---|
| `/odom` | `nav_msgs/Odometry` | `gazebo` → `pid_controller`, `slam_toolbox` |
| `/scan` | `sensor_msgs/LaserScan` | `gazebo` → `slam_toolbox` |
| `/cmd_vel` | `geometry_msgs/Twist` | `pid_controller` → `gazebo` |
| `/joint_states` | `sensor_msgs/JointState` | `gazebo` → `robot_state_publisher`, `pid_controller` |
| `/*/commands` | `std_msgs/Float64MultiArray` | `pid_controller` → `controller_manager` |
| `/map` | `nav_msgs/OccupancyGrid` | `slam_toolbox` → `rviz2` |

**To view and inspect the graph:**

```
# Visualise the full node/topic graph
ros2 run rqt_graph rqt_graph
```

```
# Echo joint states in real time
ros2 topic echo /joint_states
```

## 5. TF2 Transform Tree

The TF2 (Transform Framework 2) tree maintains spatial relationships between all coordinate frames. It is continuously published by `robot_state_publisher`, driven by `/joint_states` updates.

### 5.1 Frame Hierarchy

```
map
 └── odom                  (dynamic — calculated via diff_drive wheel encoders)
      └── base_footprint   (dynamic — 2D ground projection of the robot)
           └── dummy       (fixed joint — macro link)
                └── base_link  (robot chassis)
                     ├── left_wheel   (continuous — axis: X)
                     ├── right_wheel  (continuous — axis: X)
                     ├── nozzle_link  (revolute — axis: Y, range: ±1.5 rad)
                     └── lidar_link   (fixed — top mount, z = 0.35m)
```

### 5.2 Frame Details

| Frame | Parent Frame | Joint Type | Purpose |
|---|---|---|---|
| `map` | --- | --- | Global fixed reference frame generated by SLAM |
| `odom` | `map` | Dynamic | Odometry frame based on wheel encoders |
| `base_footprint` | `odom` | Dynamic | 2D ground projection of the robot |
| `base_link` | `dummy` | Fixed | Main physical chassis of the robot |
| `left_wheel` | `base_link` | Continuous | Left drive wheel |
| `nozzle_link` | `base_link` | Revolute | Pitch-adjustable water nozzle |
| `lidar_link` | `base_link` | Fixed | Mounting point for the 2D ray sensor |

### 5.3 TF2 in the Mapping Pipeline

When the `slam_toolbox` node initializes, it requires a continuous, unbroken TF2 chain from `odom` → `base_footprint` → `base_link` → `lidar_link`. Without this chain, laser scans cannot be mathematically projected into the global odometry frame.

**To inspect the TF2 tree during operation:**

```
# Save TF2 tree as a PDF document
ros2 run tf2_tools view_frames
```

```
# Check a specific transform manually
ros2 run tf2_ros tf2_echo odom base_footprint
```

# 6. Results and Outputs

## 6.1 System Startup Verification

After launching `gazebo.launch.py`, all components initialized successfully:

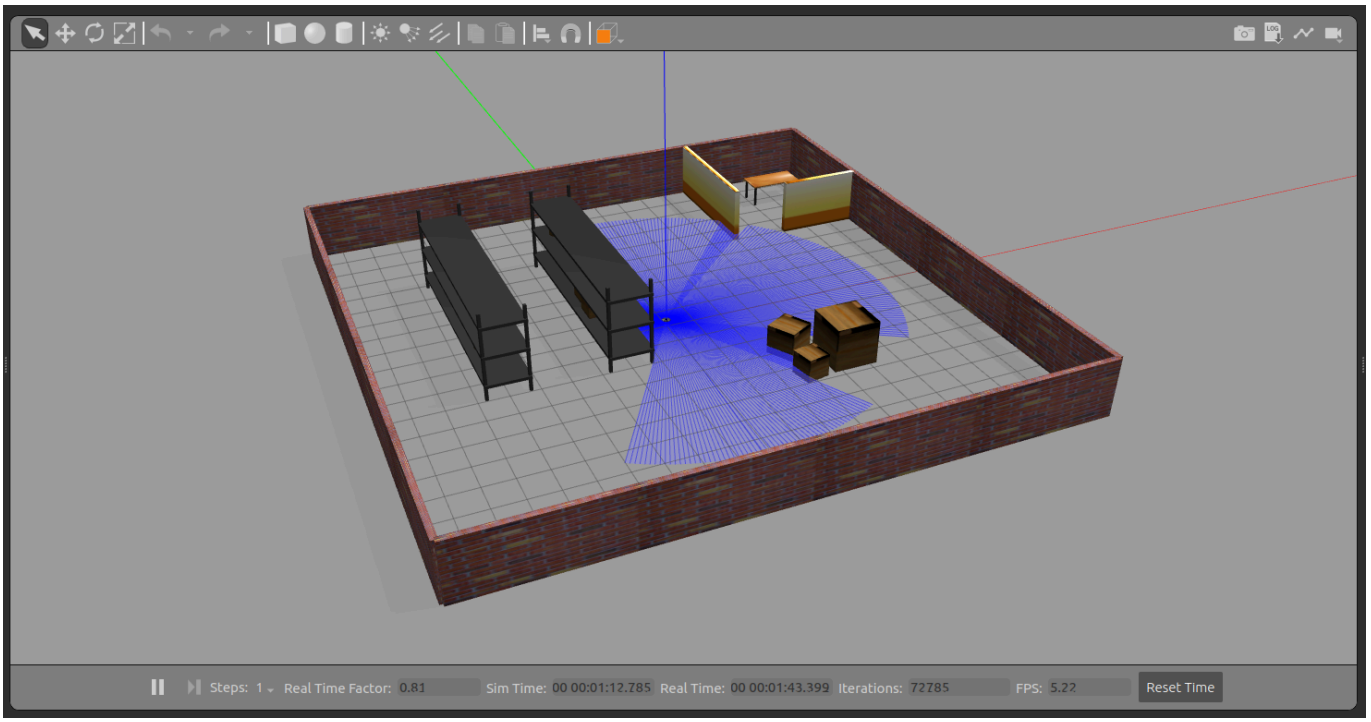
| Component | Terminal Log Confirming Success |
|---|---|
| **Gazebo Physics** | `[gazebo]: spawn_entity: Spawn status: SpawnEntity: Successfully spawned entity` |
| **diff_drive plugin** | `[gazebo]: diff_drive plugin initialized. Odometry published on /odom` |
| **joint_state_broadcaster** | `[spawner]: Configured and activated joint_state_broadcaster` |
| **nozzle_velocity_controller** | `[spawner]: Configured and activated nozzle_velocity_controller` |

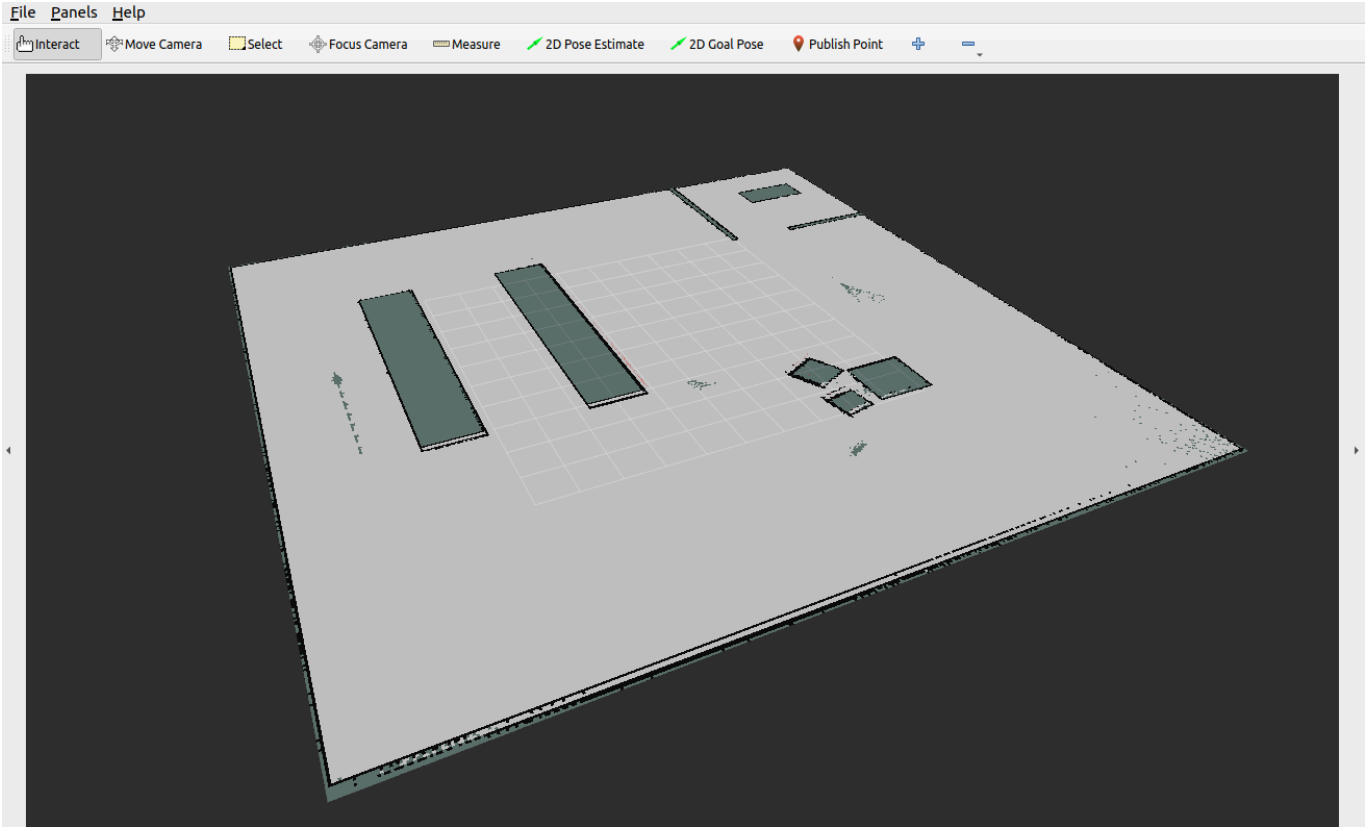
## 6.2 PID Execution Output

Terminal output from the running `pid_controller` node confirming simultaneous driving and aiming:

```
[INFO] [pid_controller]: PID Controller Node Started! Aiming for 2.0 meters
distance and 0.5 rad nozzle angle.
[INFO] [pid_controller]: [Drive] Dist: 0.23m/2.0m | Vel: 0.80
[INFO] [pid_controller]: [Drive] Dist: 1.15m/2.0m | Vel: 0.80
[INFO] [pid_controller]: [Drive] Dist: 1.82m/2.0m | Vel: 0.27
[INFO] [pid_controller]: [Drive] Dist: 1.96m/2.0m | Vel: 0.06
```

## 6.3 SLAM Mapping Output




The SLAM node successfully registered the sensor and began fusing data without dropping frames:

```
[INFO] [slam_toolbox]: Node using stack size 40000000
[INFO] [slam_toolbox]: Using solver plugin solver_plugins::CeresSolver
[INFO] [slam_toolbox]: CeresSolver: Using SCHUR_JACOBI preconditioner.
[INFO] [slam_toolbox]: Registering sensor: [lidar_link]
```

## 6.4 Issues Encountered and Resolved

| Issue | Root Cause | Resolution |
|---|---|---|
| **Physics explosion on spawn** | Self-collision enabled by default in CAD URDF export | Set `<selfCollide>false</selfCollide>` in `final_assembly.gazebo` for all links |
| **Robot spinning in place** | Left/Right wheels exported with mirrored coordinate axes | Unified joint origins and standard X-axis rotation in `final_assembly.xacro` |
| **RViz crashing (Segfault)** | VirtualBox software OpenGL rendering failed on advanced map shaders | Exported `MESA_GL_VERSION_OVERRIDE=3.3` and changed RViz scheme to `costmap` |
| **SLAM dropping laser messages** | TF and laser scan timestamps severely desynchronized | Enforced `use_sim_time:=true` for RViz and SLAM Toolbox |

## 7. Project Structure

The repository is structured into two clean ROS 2 packages separating physical description from logic:

```
ros2_ws/src/
  final_assembly_description/
  |  urdf/final_assembly.xacro            ← Robot kinematics, joints, and
geometries
  |  urdf/final_assembly.gazebo           ← Gazebo physics and control plugins
  |  launch/gazebo.launch.py              ← Complete system orchestration file
  |  launch/controller.yaml               ← ros2_control parameter configurations
  |  worlds/warehouse.world               ← Custom Gazebo 3D physical environment
  |
  firefighting_robot_control/
  |  firefighting_robot_control/
  |     pid_controller.py                 ← Dual-loop PID mathematical execution
node
```

## 8. Conclusion

This project successfully implemented a complete ROS 2 Humble software stack for a differential-drive firefighting robot. Starting from a raw CAD design, we systematically built through physical simulation (`gazebo_ros`), hardware abstraction (`ros2_control`), algorithmic logic (PID), and advanced environmental awareness (SLAM).

The key engineering challenges resolved included: rectifying physical coordinate frame mirrors from raw CAD, correctly formatting and orchestrating `ros2_control` YAML files, preventing hardware-simulation time desynchronization (`use_sim_time`), and bypassing OpenGL limitations in Virtual Machine environments. Each problem was diagnosed systematically from terminal error messages and resolved with targeted fixes.

The final system demonstrates a clean, modular architecture where each layer is independently testable. The simulation provides highly accurate odometry, the custom python node executes buttery-smooth trajectory

decay via calculus-based PID logic, and the SLAM toolbox successfully converts raw laser data into a usable operational map.

## 9. References

[1] ROS 2 Humble Documentation — https://docs.ros.org/en/humble/ [2] Gazebo ROS 2 Integration — https://gazebosim.org/docs/ [3] `ros2_control` Documentation — https://control.ros.org/humble/ [4] SLAM Toolbox Github — https://github.com/SteveMacenski/slam_toolbox